

5 Bit Carry Look Ahead Adder

VLSI Design Course Project

Bejaine C Birju

2nd year, BTech in Electronics and Communication Engineering

IIIT Hyderabad

2024102060

bejaine.birju@students.iiit.ac.in

Abstract—This report describes the design and simulation of a 5 bit Carry Look Ahead Adder using NGSpice, MAGIC and Verilog simulations. The circuit is also run on the FPGA.

Index Terms—Carry Look Ahead Adder

I. INTRODUCTION

Addition is one of the most fundamental arithmetic operations in digital signal processing and general-purpose computing. As data width and clock frequencies increase, the performance of binary adders becomes a critical bottleneck in the overall system speed. The simplest adder topology, the Ripple Carry Adder (RCA), suffers from a propagation delay that increases linearly with the number of bits ($O(N)$), making it unsuitable for high-speed applications. To mitigate this, the Carry Look-Ahead (CLA) adder is employed. The CLA architecture breaks the dependency chain by calculating the carry signals for all bit positions simultaneously, independent of the lower order bits, thereby reducing the delay complexity to logarithmic time ($O(\log N)$).

In this project, we present the design, layout, and implementation of a synchronous 5-bit Carry Look-Ahead Adder using 180nm CMOS technology. Unlike purely combinational designs, this implementation integrates storage elements, specifically True Single Phase Clock (TSPC) D Flip Flops, at both the input and output stages. This synchronous architecture ensures that inputs are sampled and stable outputs are generated within defined clock edges, a requirement for pipelined digital systems.

The design flow encompasses schematic capture and pre-layout simulation using NGSpice to verify logical correctness and timing margins, physical layout design using the MAGIC layout editor to ensure Design Rule Check (DRC) compliance, and post-layout extraction to analyze the impact of parasitic capacitances. Furthermore, the logical functionality is validated structurally using Verilog HDL, and the design is physically realized and demonstrated on an FPGA development board to verify real world operation.

II. ARCHITECTURE

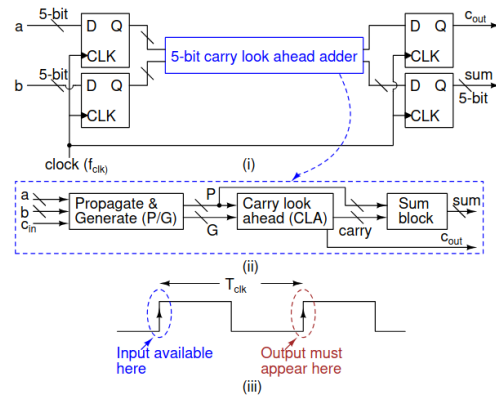


Fig. 1. Architecture of the 5 bit CLA Adder

The proposed system architecture, represents a fully synchronous computational pipeline designed to operate at high clock frequencies. The system consists of three primary stages: the Input Register Stage, the Combinational Logic Core (CLA Adder), and the Output Register Stage.

System Operation : The architecture operates on a single global clock signal (f_{clk}). At the rising edge of the clock, the 5 bit inputs $A[4:0]$ and $B[4:0]$ are sampled by the input D Flip Flops. These stable values are then passed to the CLA logic block, which computes the Sum and Carry out within the clock period (T_{clk}). The results are captured by the output D Flip Flops at the subsequent rising edge of the clock. This pipelined approach isolates the combinational delay of the adder, ensuring that glitches or transient states in the adder logic do not propagate to the next stage until the data is stable.

Design Choices and Advantages :

• Carry Look-Ahead Topology :

The core advantage of the CLA over parallel adders lies in its parallel carry generation. By generating "Propagate" (P) and "Generate" (G) signals, the CLA eliminates the ripple effect, ensuring that the delay to generate the Most Significant Bit (MSB) carry is roughly equivalent to the delay for the Least Significant Bit (LSB).

- **TSPC D Flip Flops** : The design utilizes True Single Phase Clock (TSPC) dynamic flip flops rather than standard static transmission gate registers. TSPC logic offers a reduced transistor count and eliminates the need for generating inverted clocks locally, which simplifies the clock distribution network (clock tree) and reduces skew. This topology is particularly advantageous for high speed designs due to its low clock load.
- **Static CMOS & Euler Paths** : The combinational logic gates (AND, XOR, Complex Gates) are implemented using Static CMOS logic. To optimize the physical layout, the Stick Diagram method utilizing Euler Paths is employed. This technique allows for the continuous run of diffusion strips for Logic inputs, minimizing the breaks in the diffusion layer. This results in a more compact layout area and reduced parasitic junction capacitances, directly contributing to faster switching speeds and lower power consumption.

III. DESIGN AND IMPLEMENTATION

This section details the transistor level design of the core components: the 5 bit CLA Adder and the TSPC D Flip Flop.

A. 5 Bit Carry Look Ahead Adder

The CLA adder is designed to compute the sum of two 5-bit numbers, A and B . The logic is divided into three submodules : the Propagate/Generate Generator, the Carry Generation Unit, and the Summation Unit.

1) *Propagate and Generate Logic*: For each bit position i (where $i = 0$ to 4), the propagate (P_i) and generate (G_i) signals are derived directly from the inputs according to the following equations:

$$P_i = A_i \oplus B_i \quad (1)$$

$$G_i = A_i \cdot B_i \quad (2)$$

Here, G_i indicates that a carry is generated at position i regardless of the input carry, while P_i indicates that an incoming carry will be propagated to the next position.

2) *Carry Logic Equations*: The carry for the next bit position (C_{i+1}) is calculated using the expanded boolean equations. These equations are implemented using optimized CMOS logic gates (NAND, NOR and Inverter stacks) to minimize delay.

$$C_1 = G_0 + (P_0 \cdot C_{in}) \quad (3)$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_{in} \quad (4)$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in} \quad (5)$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{in} \quad (6)$$

$$C_5 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 G_0 + P_4 P_3 P_2 P_1 P_0 C_{in} \quad (7)$$

(Note : In the physical implementation, high fan-in AND/OR terms are realized using cascaded NAND and NOR structures to preserve signal integrity and avoid the body effect penalties of long series transistor chains.)

3) *Sum Logic*: Finally, the sum bits S_i are computed by XORing the propagate signal with the carry from the previous stage:

$$S_i = P_i \oplus C_i \quad (8)$$

B. TSPC D-Flip-Flop

To achieve the synchronous requirements defined in the architecture, a Positive Edge Triggered TSPC D Flip Flop is utilized.

Circuit Description: The TSPC (True Single Phase Clock) latch operates using a single clock phase, avoiding the need for dual phase non overlapping clocks required by standard master slave latches. The circuit consists of three stages:

- 1) **Pre charge Stage**: Controlled by the clock, this stage samples the input D when the clock is low.
- 2) **Evaluation Stage**: This intermediate stage holds the data dynamically.
- 3) **Latching Stage**: On the rising edge of the clock (0 $\rightarrow$ 1 transition), the evaluation node discharges or holds charge to drive the output Q .

The use of 180nm technology with a V_{DD} of 1.8V ensures that the dynamic nodes within the TSPC latch retain their charge sufficiently during the clock period, provided the operating frequency is maintained above the leakage threshold. This design provides a high speed, glitch free output suitable for driving the subsequent CLA logic stages.

IV. NGSPICE PRE LAYOUT IMPLEMENTATION

A. D Flip Flop

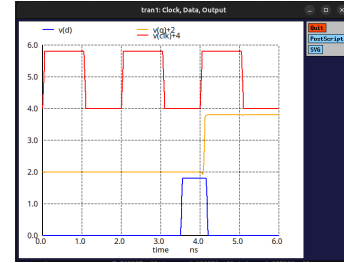


Fig. 2. Waveforms obtained for the pre layout simulation of the TSPC DFF

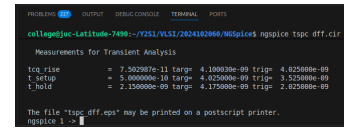


Fig. 3. Clock to Q delay

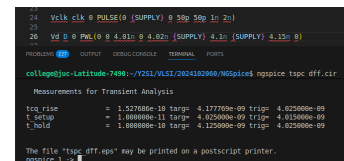


Fig. 4. Setup Time

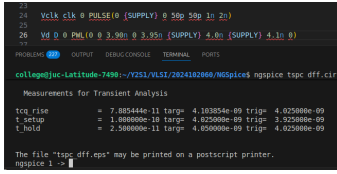


Fig. 5. Hold time

TABLE I
NGSPICE MEASURED TIMING PARAMETERS

Parameter	Value
Clock to Q delay	7.592987×10^{-11}
Setup time	1.00000×10^{-11}
Hold time	2.50000×10^{-11}

B. CLA

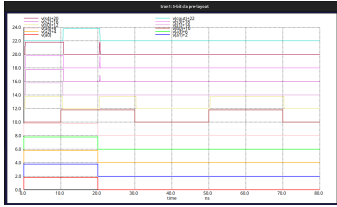


Fig. 6. CLA waveforms

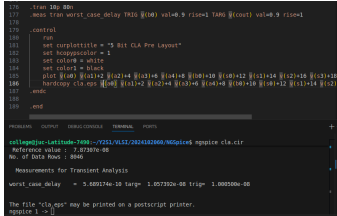


Fig. 7. CLA Delay obtained from the Pre Layout NGSpice simulation

$$\text{Frequency of the CLA Adder} = f = \frac{1}{5 \times 10^{-10}} = 2 \text{GHz}$$

C. Full Circuit : CLA with TSPC Flip Flops

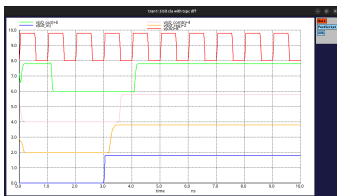


Fig. 8. CLA with TSPC D Flip Flops Pre Layout NGSpice simulated waveforms

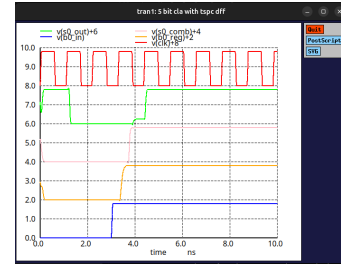


Fig. 9. Noticeable variation in circuit operation at a clock speed of 1.09ns

Thus the maximum clock speed for accurate operation is $T_{clk} = 1.09 \text{ns}$

V. MAGIC IMPLEMENTATION

A. Stick Diagrams

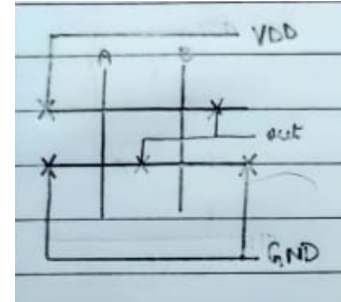


Fig. 10. 2 input NOR gate

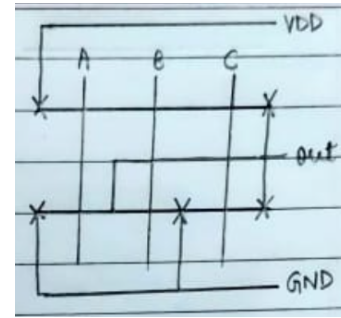


Fig. 11. 3 input NOR gate

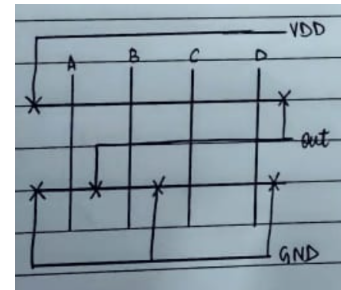


Fig. 12. 4 input NOR gate

B. MAGIC Layouts

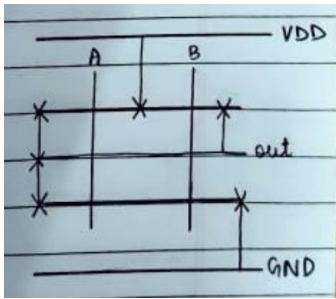


Fig. 13. 2 input NAND gate

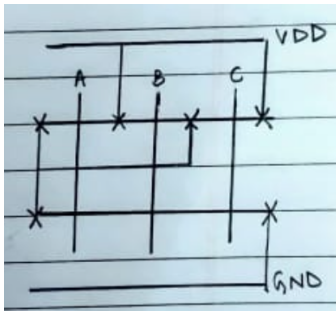


Fig. 14. 3 input NAND gate

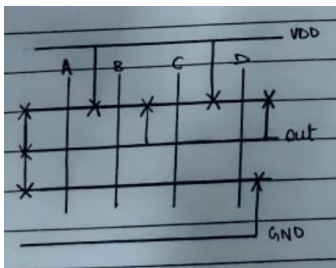


Fig. 15. 4 input NAND gate

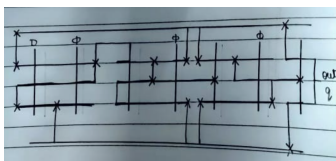


Fig. 16. TSPC D Flip Flop

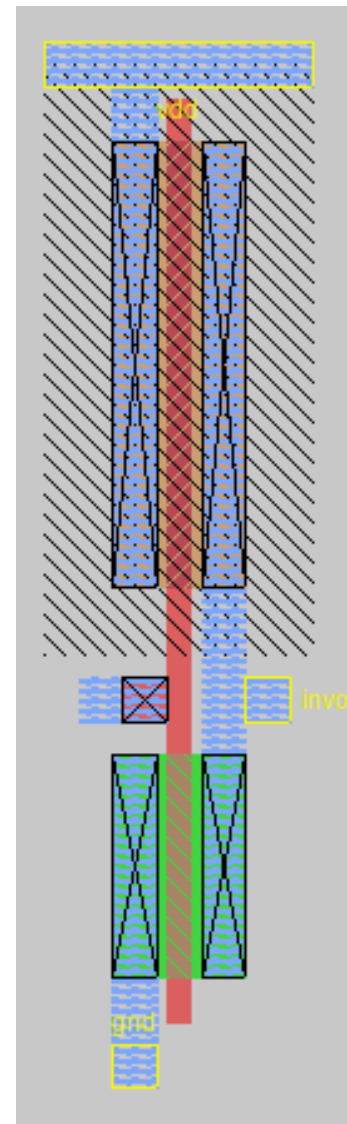


Fig. 17. Inverter MAGIC Layout

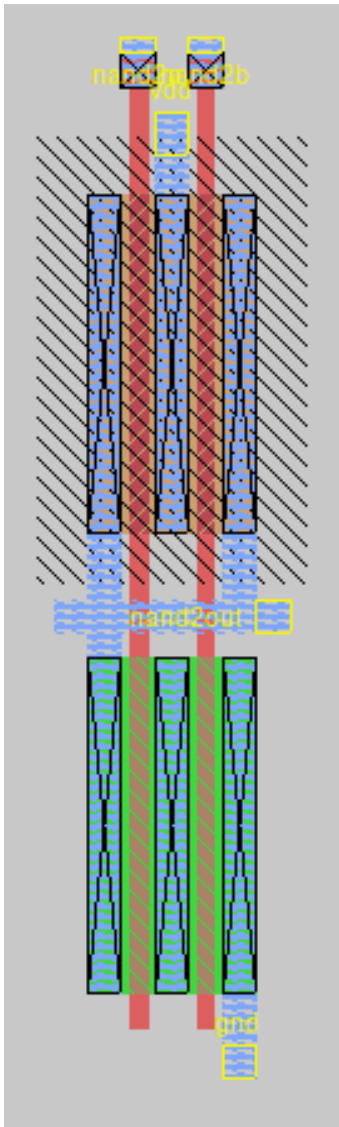


Fig. 18. 2 input NAND MAGIC Layout

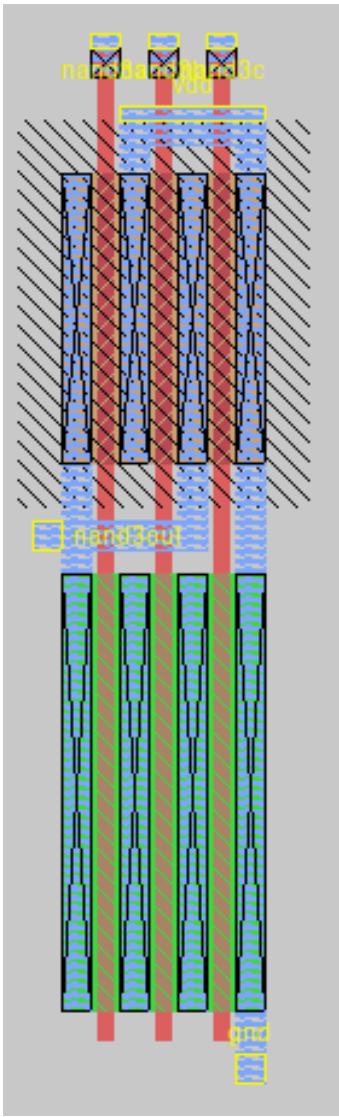


Fig. 19. 3 input NAND MAGIC Layout

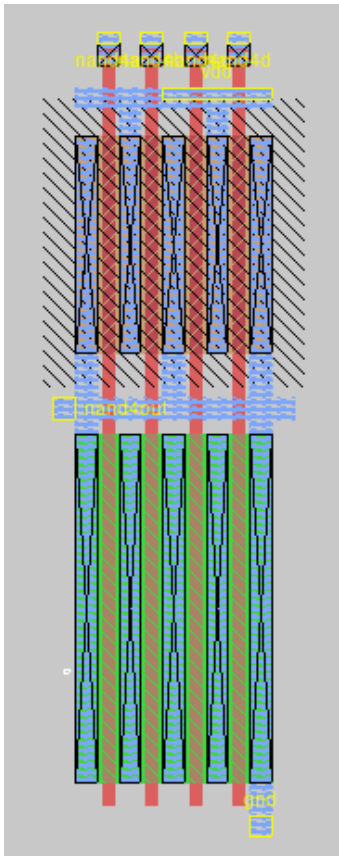


Fig. 20. 4 input NAND MAGIC Layout

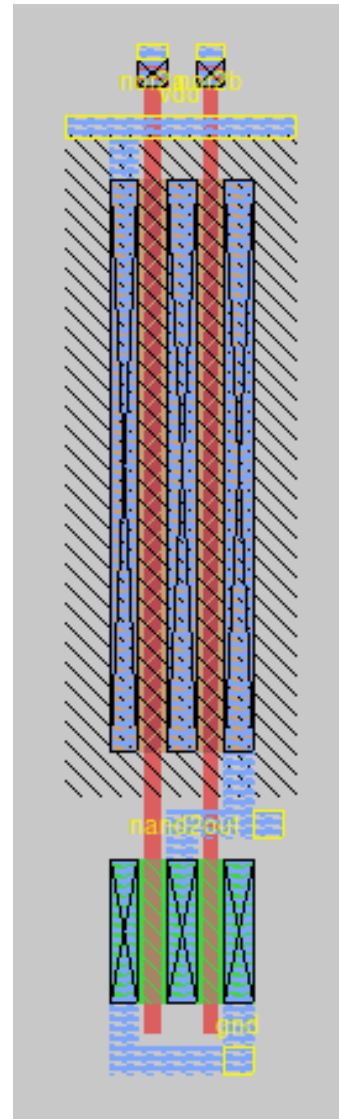


Fig. 21. 2 input NOR MAGIC Layout

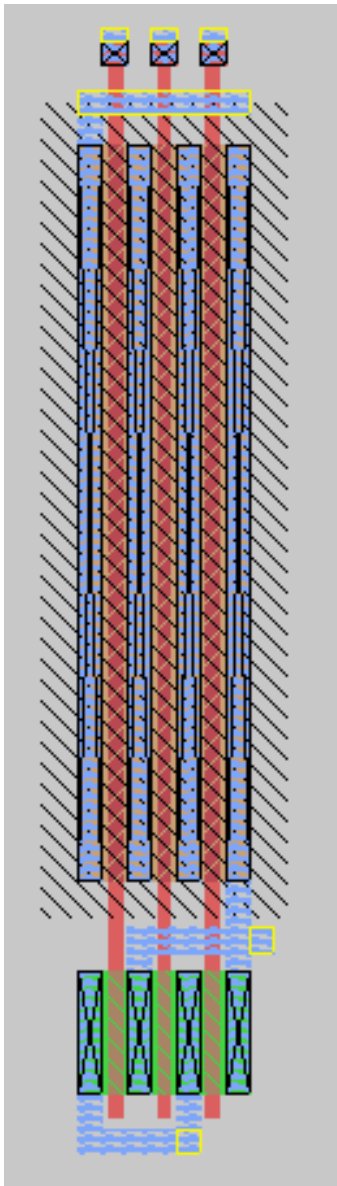


Fig. 22. 3 input NOR MAGIC Layout

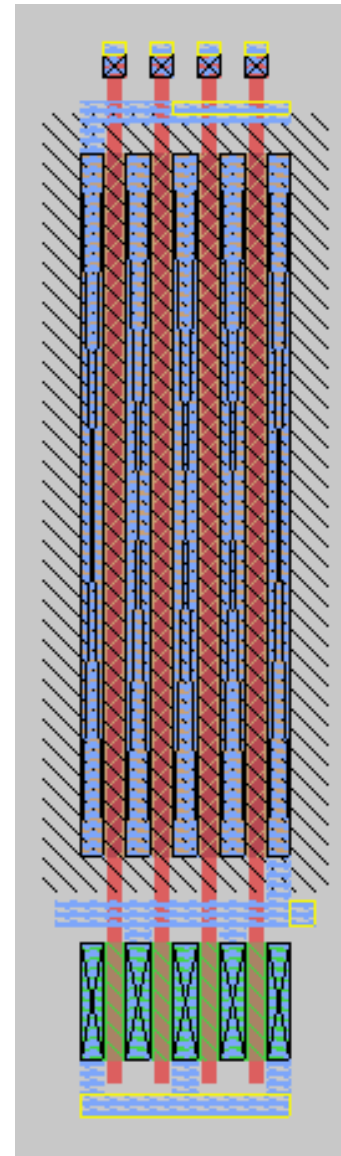


Fig. 23. 4 input NOR MAGIC Layout

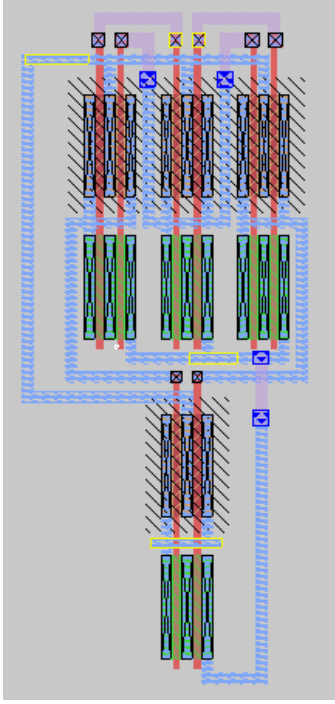


Fig. 24. XOR MAGIC Layout

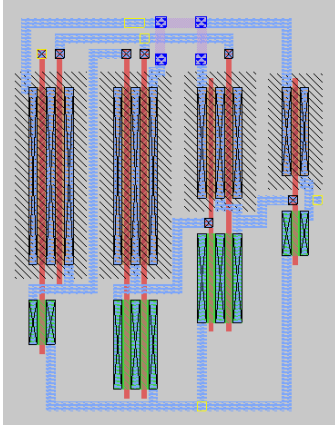


Fig. 25. TSPC DFF MAGIC Layout

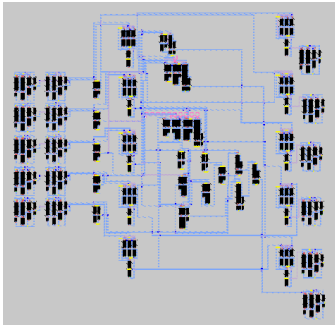


Fig. 26. MAGIC Layout of the CLA adder with the TSPC Flip Flops

C. NGSpice Post layout Implementation

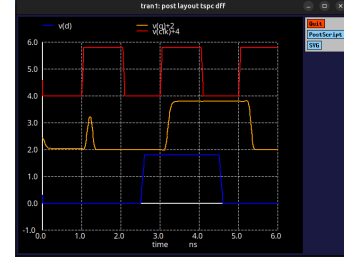


Fig. 27. Waveforms obtained upon checking the post layout TSPC D Flip Flop from MAGIC Layout extraction

VI. VERILOG IMPLEMENTATION

This section details the implementation of a **5 bit Carry Lookahead Adder (CLA)** using Verilog HDL. The design incorporates input and output flip flops to create a **pipelined architecture**. This structure ensures high speed operation by isolating the combinational delay of the carry logic between register stages. Unlike Ripple Carry Adders, the CLA architecture precomputes carry signals in parallel, significantly reducing propagation delay as bit width increases.

A. Module wise Breakdown

1) *Input and Output Declarations:* The top level module defines the interface for the 5 bit addition.

```
1 input wire [4:0] A_in, B_in;
2 input wire Cin, clk, rst;
3 output wire [4:0] Sum_out;
4 output wire Cout_out;
```

- **Inputs:** Two 5 bit operands (A_in, B_in), an external carry in (Cin), a system clock (clk), and an active low reset (rst).
- **Outputs:** A 5 bit result (Sum_out) and a final carry out bit (Cout_out).

2) *Internal Register Declarations:* The following is the code

```
1 wire [4:0] A_reg, B_reg;
```

- These wires connect the output of the input flip flops to the combinational CLA logic, ensuring stable signals are presented to the adder core during the clock cycle.

3) *Input Flip-Flop Instantiation:* To handle the 5 bit width efficiently, a generate block is utilized to instantiate D flip flops for every bit position.

```
1 generate
2   for (i=0; i<5; i=i+1) begin
3     d_flip_flop ff_a (
4       .Q(A_reg[i]), .D(A_in[i]), ...);
5   end
6 endgenerate
```

- This stage captures the asynchronous inputs on the rising edge of the clock, synchronizing the data before processing.

4) *Generate (G) and Propagate (P) Signals:* Inside the combinational logic module, the generate and propagate signals are computed in parallel dataflow logic:

```
1 assign P = A ^ B;
2 assign G = A & B;
```

- **Generate (G):** High if a column generates a carry regardless of input carry ($A \cdot B$).
- **Propagate (P):** High if a column propagates an incoming carry ($A \oplus B$).

5) *Carry Look-Ahead Logic:* The carry for each bit position is derived directly from inputs and the Carry-in, flattening the logic depth.

```
1 assign C[0] = Cin;
2 assign C[1] = G[0] | (P[0] & C[0]);
3 ...
4 assign C[5] = G[4] | (P[4] & G[3]) ... ;
```

- The carry chain ($C[0]$ to $C[5]$) is calculated simultaneously. $C[5]$ represents the final overflow/carry-out of the 5-bit operation.

6) *Sum Calculation:* The sum is obtained using :

```
1 assign Sum = P ^ C[4:0];
```

- The final sum is computed by XORing the propagate signal with the carry in of the respective bit position.

7) *Output Flip-Flops:* The results from the combinational logic are registered before leaving the module.

```
1 d_flip_flop ff_sum (.Q(Sum_out[i]),
2 .D(Sum_comb[i]), ...);
3 d_flip_flop ff_cout (.Q(Cout_out),
4 .D(Cout_comb), ...);
```

- This creates a fully synchronous interface, isolating the critical path of the adder from the external downstream logic.

B. Verilog Waveforms Visualisation

The implemented 5 bit CLA adder utilizes a pipelined data-path to maximize throughput. By separating input acquisition, logic computation, and output driving into distinct register stages, the design is robust and suitable for high frequency digital systems. The use of `generate` statements ensures the code remains scalable and concise.

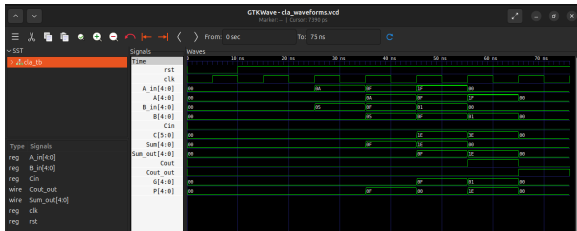


Fig. 28. Simulation Waveform showing 5 bit addition latency

VII. VIVADO & FPGA IMPLEMENTATION

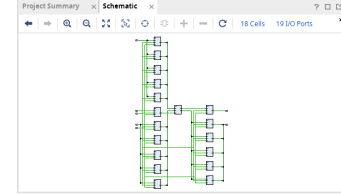


Fig. 29. Elaborated Design obtained on Vivado upon performing RTL Analysis

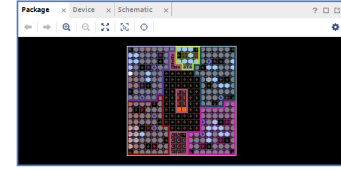


Fig. 30. Package Layout in Vivado

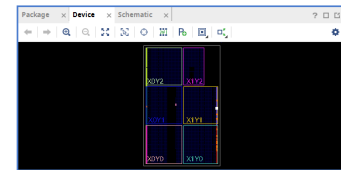


Fig. 31. Device Layout in Vivado

Upon generating the bitstream using Xilinx Vivado and dumping the bitstream onto the FPGA, we obtain the correct



Fig. 32. Digital Readings obtained on the FPGA

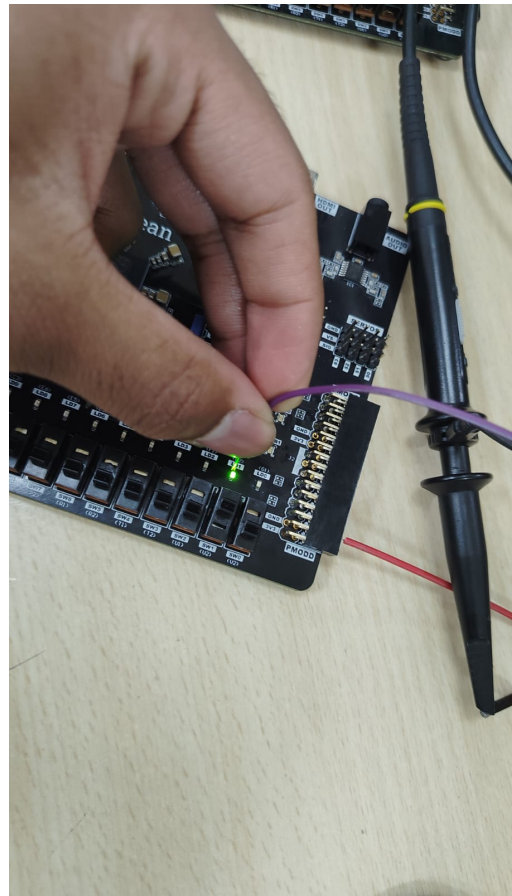


Fig. 33. Readings taken using a wire connected to the pin of the LED



Fig. 34. Analog output obtained on the oscilloscope